Programming essentials in C++ _ 2nd

Presented by: Dr. Asmaa Saad

Table of Content

Contents

Programming essentials in C++ _ 2nd 1

Chapter 1: Introduction to C++ 5

1. 1 Environment Setup 6

2 The Problem Analysis–Coding–Execution Cycle 9

3. Understanding Different Types of Errors in C++ Programming 10

Chapter 2: C BASIC SYNTAX 14

2.1 Tokens in C++ 14

2.1 DATA TYPES 21

Chapter 3 26

3.1 Variables 26

3.2 const Keyword 35

3.3 Introduction to strings 36

Chapter 4: Operators 38

4.1 Basic Input/Output 46

Chapter 5: Control Structures 54

5.1 C++ Exceptions 67

Chapter 6: Functions 72

Chapter 7: ARRAYS 79

Chapter 8: FILE I/O 92
8.1 C++ Files 92
Chapter 9: Object Oriented Programming 94
9.1 C++ Classes and Objects 95
9.2 Constructor 100
9.3 Encapsulation 105
9.4 Inheritance 107
9.5 Polymorphism 112
Chapter 1 Summary and Exercises 116
Review Questions 116
Practice and Programming Exercises 117
Chapter 2 Summary and Exercises 120
Review Questions 120
Practice and Programming Exercises 121
Chapter 3 Summary and Exercises 124
Review Questions 124
Practice and Programming Exercises 125
Chapter 4 Summary and Exercises 128
Review Questions 128
Practice and Programming Exercises 129
Chapter 5 Summary and Exercises 132
Review Questions 132

Practice and Programming Exercises 133
Chapter 6 Summary and Exercises 136
Review Questions 136
Practice and Programming Exercises 137
Chapter 7 Summary and Exercises 140
Review Questions 140
Practice and Programming Exercises 141
Chapter 8 Summary and Exercises 144
Review Questions 144
Practice and Programming Exercises 145
Chapter 9 Summary and Exercises 148
Review Questions 148
Practice and Programming Exercises 149
References 152

Chapter 1: Introduction to C++

This book introduces the C++ programming language. C++ is a statically typed, compiled, general-purpose, case-sensitive, free-form programming language that supports procedural, object-oriented, and generic programming. C++ is regarded as a middle-level language, as it comprises a combination of both high-level and low-level language features. C++ was developed by Bjarne Stroustrup starting in 1979 at Bell Labs in Murray Hill, New Jersey, as an enhancement to the C language and originally named C with Classes but later it was renamed C++ in 1983. C++ is a superset of C, and that virtually any legal C program is a legal C++ program.

C++ fully supports object-oriented programming, including the four pillars of object-oriented development:

- Encapsulation
- Data hiding
- Inheritance
- Polymorphism

Standard Libraries

Standard C++ consists of three important parts:

- The core language giving all the building blocks including variables, data types and literals, etc.
- The C++ Standard Library giving a rich set of functions manipulating files, strings, etc.
- The Standard Template Library giving a rich set of methods manipulating data structures, etc.

1. 1 Environment Setup

An IDE (Integrated Development Environment) is used to edit AND compile the code. Popular IDE's include Code::Blocks, Eclipse, and Visual Studio. These are all free, and they can be used to both edit and debug C++ code. You can find the latest version of Codeblocks at http://www.codeblocks.org/.

Let's create our first C++ file. Open Codeblocks and go to **File > New > Empty File**. Write the following C++ code and save the file as myfirstprogram.cpp (**File > Save File as**):

```
#include <iostream>
using namespace std;

int main()
{
   cout << "Hello World!";
   return 0;
}
```

Then, go to Build > Build and Run to run (execute) the program. The result will look something to this:
Hello World!

Another Program

Sample program

```
#include <iostream>
using namespace std;
int main()
{
cout << "Enjoy yourself with C++!" <<< endl;
return 0;
}
```

Screen output

Enjoy yourself with C++!

Structure of function main ()

Function name

Type of function —>|int main()

Beginning of —|{

function

What the program does ,

(statements) Function block

End of function —|}

How to Download and Install CodeBlocks for C/C++ Programming?

CodeBlocks is an open-source, cross-platform (Windows, Linux, MacOS), and free C/C++ IDE. It supports many compilers, such as GNU GCC (MinGW and Cygwin) and MS Visual C++. It supports interactive debugging (via GNU GDB or MS CDB). CodeBlocks is surprisingly versatile, and in my opinion, much better than the Visual Studio suite. The mother site of CodeBlocks is www.codeblocks.org.

Step 1: Download

Goto http://www.codeblocks.org/downloads. Click "Download the binary release". Select your operating platform (e.g., Windows 2000/XP/Vista/7). Download the installer with GCC Compiler, e.g., codeblocks-13.12mingw-setup.exe (98 MB) (which includes MinGW's GNU GCC compiler and GNU GDB debugger).

Step 2: Install

Run the downloaded installer. Accept the default options.

A program written in a high-level language is called a *source program* or *source code*. Because a computer cannot understand a source program, a source program must be translated into machine code for execution. The translation can be done using another programming tool called an *interpreter* or a *compiler*.

- An interpreter reads one statement from the source code, translates it to the machine code or virtual machine code, and then executes it right away.
- A compiler translates the entire source code into a machine-code file, and the machine code file is then executed.

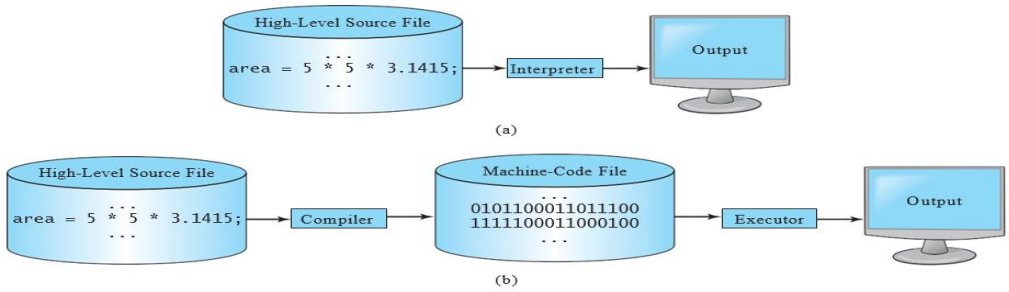


Figure 2 An interpreter translates and executes a program one statement at a time. (b) A compiler

translates the entire source program into a machine-language file for execution.

2 The Problem Analysis–Coding–Execution Cycle

Problem Analysis

Using Any Algorithm Design

Programming

Language

Coding

Complier

'Translate Code Errors

To Machine

Language Execution

Errors

B..

— a=

Think Write Code

3. Understanding Different Types of Errors in C++ Programming

When programming in C++, you may come across different types of errors. These errors can generally be categorized into three main types: syntax errors, semantic errors, and runtime errors. It is essential to be aware of these errors, so you can tackle them effectively when they occur. In the following sections, we will explore each error type in greater detail.

1. **Syntax errors:** Errors that occur when you violate the rules of writing C++ syntax are known as syntax errors. This compiler error indicates something that must be fixed before the code can be compiled. All these errors are detected by compiler and thus are known as compile-time errors.

   Most frequent syntax errors are:

   - Missing Parenthesis (})
   - Printing the value of variable without declaring it
   - Missing semicolon like this:

```
// C++ program to illustrate
// syntax error
#include <iostream>
using namespace std;
```

```
void main()
{
int x = 10;
int y = 15;
cout << " "<< (x, y) // semicolon missed
}
```

Output:

```
error: expected ';' before '}' token
```

2. **Run-time Errors :** Errors which occur during program execution(run-time) after successful compilation are called run-time errors. One of the most common run-time errors is division by zero also known as Division error. These types of error are hard to find as the compiler doesn't point to the line at which the error occurs. For more understanding run the example given below.

```
// C++ program to illustrate
// run-time error
#include <iostream>
using namespace std;

int main()
{
   int n = 9, div = 0;
   // wrong logic
   // number is divided by 0,
```

```
    // so this program abnormally terminates
    div = n / 0;
    cout << "result = " << div;
    return 0;
}
```

Output:

```
warning: division by zero [-Wdiv-by-zero]
div = n/@;
```

3. **Logical Errors :** On compilation and execution of a program, desired output is not obtained when certain input values are given. These types of errors which provide incorrect output but appears to be error free are called logical errors. These are one of the most common errors done by beginners of programming. These errors solely depend on the logical thinking of the programmer and are easy to detect if we follow the line of execution and determine why the program takes that path of execution.

```
// C++ program to illustrate
// logical error
#include <iostream>
using namespace std;

int main()
{
    int i = 0;
    // logical error: a semicolon after loop
    for (i = 0; i < 3; i++);
```

```
    {
        cout << "loop";
        continue;
    }
    return 0;
}
```

No output

Chapter 2: C BASIC SYNTAX

2.1 Tokens in C++

C++ Program Structure

Let us look at a simple code that would print the words Hello World.

```
#include <iostream>
using namespace std;

// main() is where program execution begins.
int main()
{
   cout << "Hello World"; // prints Hello World
   return 0;
}
```

Let us look at the various parts of the above program:

1. The C++ language defines several headers, which contain information that is either necessary or useful to your program. For this program, the header <iostream> is needed.

2. The line using namespace std; tells the compiler to use the std namespace. Namespaces are a relatively recent addition to C++.

3. The next line'// main() is where program execution begins.' is a single-line comment available in C++. Single-line comments begin with // and stop at the end of the line.

4. The line int main() is the main function where program execution begins.

5. The next line cout << "This is my first C++ program."; causes the message "This is my first C++ program" to be displayed on the screen.

6. The next line return 0; terminates main() function and causes it to return the value 0 to the calling process.

Compile & Execute C++ Program

Let's look at how to save the file, compile and run the program. Please follow the steps given
below:

1. Open a text editor and add the code as above.

2. Save the file as: hello.cpp

3. Open a command prompt and go to the directory where you saved the file.

4. Type 'g++ hello.cpp' and press enter to compile your code. If there are no errors in your code the command prompt will take you to the next line and would generate a.out executable file.

5. Now, type 'a.out' to run your program.

6. You will be able to see ' Hello World ' printed on the window.

$ g++ hello.cpp

$ ./a-out

Hello World
Make sure that g++ is in your path and that you are running it in the directory containing file
hello.cpp.
You can compile C/C++ programs using makefile. For more details, you can check our 'Makefile Tutorial'.

Semicolons & Blocks in C++

In C++, the semicolon is a statement terminator. That is, each individual statement must

be
ended with a semicolon. It indicates the end of one logical entity.

For example, following are three different statements:

```
x = y;
y = y + 3;
add(x, y);
```

A block is a set of logically connected statements that are surrounded by opening and closing braces. For example:

```
{
   cout << "Hello World"; // prints Hello World
   return 0;
}
```

C++ does not recognize the end of the line as a terminator. For this reason, it does not matter where you put a statement in a line. For example:

```
x = y;
y = y + 3;
add(x, y);
```

is the same as

```
x = y; y = y + 3; add(x, y);
```

C++ Identifiers

C++ Identifiers

A C++ identifier is a name used to identify a variable, function, class, module, or any other

user-defined item. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores, and digits (0 to 9).

C++ does not allow punctuation characters such as @, $, and % within identifiers. C++ is a

case-sensitive programming language. Thus, Manpower and manpower are two different identifiers in C++.

Here are some examples of acceptable identifiers:

mohd zara abc_— move_name_ a_123

myname50 temp = j a23b9 retVal

C++ Keywords

The following list shows the reserved words in C++. These reserved words may not be used

as constant or variable or any other identifier names.

asm else new this

auto enum operator throw

bool explicit private true

break export protected try

case extern public typedef
catch false register typeid
char float reinterpret_cast typename
class for return union
const friend short unsigned
const_cast goto signed using
continue if sizeof virtual
default inline static void
delete int static_cast volatile
do long struct wchar_t
double mutable switch while
dynamic_cast namespace template

Whitespace in C++

A line containing only whitespace, possibly with a comment, is known as a blank line, and
C++ compiler totally ignores it.

Whitespace is the term used in C++ to describe blanks, tabs, newline characters and comments. Whitespace separates one part of a statement from another and enables the

compiler to identify where one element in a statement, such as int, ends and the next element
begins. Statement 1:

In the above statement there must be at least one whitespace character (usually a space) between int and age for the compiler to be able to distinguish them. Statement 2:

fruit = apples + oranges; // Get the total fruit

In the above statement 2, no whitespace characters are necessary between fruit and =, or between = and apples, although you are free to include some if you wish for readability purpose.

COMMENTS IN C++

Program comments are explanatory statements that you can include in the C++ code. These comments help anyone reading the source code. All programming languages allow for some form of comments.

C++ supports single-line and multi-line comments. All characters available inside any comment are ignored by the C++ compiler.

C++ comments start with /* and end with */. For example:

```
/* This is a comment */

/* C++ comments can also
   span multiple lines */
```

A comment can also start with //, extending to the end of the line.

```
// Comment out printing of Hello World:
// cout << "Hello World"; // prints Hello World
```

2.1 DATA TYPES

While writing program in any language, you need to use various variables to store various
information. Variables are nothing but reserved memory locations to store values. This means
that when you create a variable you reserve some space in memory.
You may like to store information of various data types like character, wide character, integer,
floating point, double floating point, boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.
Primitive Built-in Types
C++ offers the programmer a rich assortment of built-in as well as user defined data types.

Following table lists down seven basic C++ data types:

Type Keyword

Boolean bool

Character char

Integer int

Floating point float

Double floating point double

Valueless void

Wide character wchar_t

Several of the basic types can be modified using one or more of these type modifiers:

«signed

«unsigned

«short

- long

The following table shows the variable type, how much memory it takes to store the value in
memory, and what is maximum and minimum value which can be stored in such type of variables.

Type Typical Bit Width Typical Range

char byte -127 to 127 or 0 to 255

unsigned char byte 0 to 255

signed char byte -127 to 127

int bytes -2147483648 to 2147483647

unsigned int abytes 0 to 4294967295

signed int bytes -2147483648 to 2147483647

short int 2bytes -32768 to 32767

unsigned short int Range 0 to 65,535

signed short int Range -32768 to 32767

long int bytes -2,147,483,647 to 2,147,483,647

signed long int bytes same as long int

unsigned long int bytes 0 to 4,294,967,295

float bytes +/- 3.48 +/- 38 (~7 digits)

double 8bytes 4/- 1.7e +/- 308 (~15 digits)

long double 8bytes 4/- 1.7e +/- 308 (~15 digits)

wcehar_t 2 or 4 bytes 1 wide character
The size of variables might be different from those shown in the above table, depending on
the compiler and the computer you are using.
Following is the example, which will produce correct size of various data types on your computer.

wchar_t 2 or 4 bytes 1 wide character
The size of variables might be different from those shown in the above table, depending on
the compiler and the computer you are using,
Following is the example, which will produce correct size of various data types on your
computer.

```
#include <iostream>
using namespace std;
int main()
```

```
{
cout << "Size of char : " << sizeof(char) <<< endl;
cout << "Size of int : " << sizeof(int) <<< endl;
cout << "Size of short int : " << sizeof(short int) << endl;
cout << "Size of long int : * << sizeof(long int) <<< endl;
cout << "Size of float : " << sizeof(float) <<< endl;
cout << "Size of double : * << sizeof(double) <<< endl;
cout << "Size of wchar_t : " << sizeof(wchar_t) <<< endl;
return 0;
}
```

This example uses endl, which inserts a new-line character after every line and << operator
is being used to pass multiple values out to the screen. We are also using sizeof() function
to get size of various data types.

When the above code is compiled and executed, it produces the following result which can
vary from machine to machine:

```
Size of char : 1
Size of int : 4
Size of short int : 2
Size of long int : 4
Size of float : 4
Size of double : 8
Size of wchar_t : 4
```

Chapter 3

3.1 Variables

A variable provides us with named storage that our programs can manipulate. Each variable
in C++ has a specific type, which determines the size and layout of the variable's memory;
the range of values that can be stored within that memory; and the set of operations that

can
be applied to the variable.

The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C++ is case-sensitive:

There are following basic types of variable in C++ as explained in last chapter:

A variable provides us with named storage that our programs can manipulate. Each variable
in C++ has a specific type, which determines the size and layout of the variable's memory;
the range of values that can be stored within that memory; and the set of operations that can
be applied to the variable.
The name of a variable can be composed of letters, digits, and the underscore character. It
must begin with either a letter or an underscore. Upper and lowercase letters are distinct
because C++ is case-sensitive:
There are following basic types of variable in C++ as explained in last chapter:

Type Description

bool Stores either value true or false.

char Typically a single octet (one byte). This is an integer type.

int The most natural size of integer for the machine.

float A single-precision floating point value.

double 'A double-precision floating point value.

void Represents the absence of type.

wchar_t 'A wide character type.
C++ also allows to define various other types of variables, which we will cover in subsequent
chapters like Enumeration, Pointer, Array, Reference, Data structures, and Classes.
Following section will cover how to define, declare and use various types of variables.

Variable Definition in C++
A variable definition tells the compiler where and how much storage to create for the variable.
A variable definition specifies a data type, and contains a list of one or more variables of that
type as follows:
Here, type must be a valid C++ data type including char, w_char, int, float, double, bool or
any user-defined object, etc., and variable_list may consist of one or more identifier names
separated by commas. Some valid declarations are shown here:

int a, i, ks

char c, ch;

float f, salary;

double d;

The line int i, j, k; both declares and defines the variables i, j and k; which instructs the
compiler to create variables named i, j and k of type int.

Variables can be initialized (assigned an initial value) in their declaration. The initializer
consists of an equal sign followed by a constant expression as follows:

The line int i, j, k; both declares and defines the variables i, j and k; which instructs the compiler to create variables named i, j and k of type int.
Variables can be initialized (assigned an initial value) in their declaration. The initializer consists of an equal sign followed by a constant expression as follows:
Some examples are:

extern int d= 3, f= 5; // declaration of d and f.

int d= 3, f = 53 // definition and initializing d and f.

byte z = 22; // definition and initializes z.

char x = 'x'; // the variable x has the value 'x'.

For definition without an initializer: variables with static storage duration are implicitly initialized with NULL (all bytes have the value 0); the initial value of all other variables is
undefined.

Variable Declaration in C++

A variable declaration provides assurance to the compiler that there is one variable existing
with the given type and name so that compiler proceed for further compilation without needing complete detail about the variable. A variable declaration has its meaning at the time
of compilation only, compiler needs actual variable declaration at the time of linking of the
program.

A variable declaration is useful when you are using multiple files and you define your variable
in one of the files which will be available at the time of linking of the program. You will
use extern keyword to declare a variable at any place. Though you can declare a variable
multiple times in your C++ program, but it can be defined only once in a file, a function or a
block of code.

Example

Try the following example where a variable has been declared at the top, but it has been

defined inside the main function:

```
#include <iostream>
using namespace std;
/1 Variable declaration:
extern int a, bs
extern int c;
extern float ;
int main ()
{
/1 Variable definition:
int a, b;
int 5
float f;
// actual initialization
a= 10;
b = 20;
c=atbs
cout << c <<< endl 5
¥ = 70.0/3.0;
cout << f <<< endl ;


return 0;
+
```

When the above code is compiled and executed, it produces the following result:

3e

23.3333

Variable Scope

A scope is a region of the program and broadly speaking there are three places, where variables can be declared:

- Inside a function or a block which is called local variables,
- In the definition of function parameters which is called formal parameters.
- Outside of all functions which is called global variables.

Local Variables

Variables that are declared inside a function or block are local variables. They can be used

only by statements that are inside that function or block of code. Local variables are not

known to functions outside their own. Following is the example using local variables:

```
#include <iostream>
using namespace std;
int main()
{
// Local variable declaration:
int a, b;
int c;
```

Local Variables

Variables that are declared inside a function or block are local variables. They can be used

only by statements that are inside that function or block of code. Local variables are not
known to functions outside their own. Following is the example using local variables:

```
#include <iostream>
using namespace std;
int main ()
{
// Local variable declaration:
int a, b;
int c;
// actual initialization
a = 10;
b = 20;
c = a + b;
cout << c;
return 0;
    }
```

Global Variables

Global variables are defined outside of all the functions, usually on top of the program. The global variables will hold their value throughout the life-time of your program. A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration. Following is the example using global and local variables:

Global variables are defined outside of all the functions, usually on top of the program. The

global variables will hold their value throughout the life-time of your program.
A global variable can be accessed by any function. That is, a global variable is available for
use throughout your entire program after its declaration. Following is the example using global
and local variables:|

```
#include <iostream>
using namespace std;
// Global variable declaration:
int g;
int main ()
{
// Local variable declaration:
int a, b;
// actual initialization
a= 10;
b = 20;
g = a + b;
cout << g;
return 0;
}
```

A program can have same name for local and global variables but value of local variable inside
a function will take preference. For example:

```
#include <iostream>
using namespace std;
```

```
// Global variable declaration:
int g = 20;
int main ()
{
// Local variable declaration:
int g = 10;
cout << g;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

3.2 const Keyword

You can use const prefix to declare constants with a specific type as follows:

```
const type variable = value;
```

Following example explains it in detail:

```
#include <iostream>
using namespace std;
int main()
{
   const int LENGTH = 10;
   const int WIDTH = 5;
   const char NEWLINE = '\n';
   int area;

area = LENGTH * WIDTH;
cout << area;
```

```
cout << NEWLINE;
return 0;
}
```

When the above code is compiled and executed, it produces the following result:

Character and string literals have certain peculiarities, like the escape codes. These are special characters that are difficult or impossible to express otherwise in the source code of a program, like newline (\n) or tab (\t). All of them are preceded by a backslash (\). Here you have a list of some of such escape codes:

[ees
[eacspece |
[sform eed (oan feed
[nate quote |
[rule ote CY]
[laueston mark =|
[Noadeslash GQ)

3.3 Introduction to strings

Variables that can store non-numerical values that are longer than one single character are known as strings. The C++ language library provides support for

strings through the standard string class. This is not a fundamental type, but it behaves in a similar way as fundamental types do in its most basic usage.

A first difference with fundamental data types is that in order to declare and use objects (variables) of this type we
need to include an additional header file in our source code: <string> and have access to the std namespace
(which we already had in all our previous programs thanks to the using namespace statement).
my first string This is a string

```
#include <iostream>
#include <string>
using namespace std;
int main ()
{
string mystring = "This is a string";
cout << mystring;
return 0;
}
```

As you may see in the previous example, strings can be initialized with any valid string literal just like numerical
type variables can be initialized to any valid numerical literal. Both initialization formats are valid with strings:

```
string mystring = "This is a string";
string mystring ("This is a string");
```

Chapter 4: Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations. C++ is rich in built-in operators and provide the following types of operators:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators

Arithmetic Operators

There are following arithmetic operators supported by C++ language:

Assume variable A holds 10 and variable B holds 20, then:

Operator Description Example

+ Adds two operands A+ Bwill give 30

- Subtracts second operand from the A - Bwill give -10
first

* Multiplies both operands A * Bwill give 200

/ Divides numerator by de-numerator B/ Awill give 2

% Modulus Operator and remainder of B % A will give 0
after an integer division

++ Increment operator, increases A++ will give 11
integer value by one

-- Decrement operator, decreases A-- will give 9
integer value by one

For example:

```
#include <iostream>

using namespace std;

main()

{
int a = 21;
int b = 10;
int c;
c=a+b;
cout << "Line 1 - Value of c is :" << c <<< endl ;
c=a-b;
cout << "Line 2 - Value of c is :" << c <<< endl ;
c=a*b;
cout << "Line 3 - Value of c is :" << c <<< endl ;
c=a/b;
cout << "Line 4 - Value of c is :" << c <<< endl ;
c=a%b;
cout << "Line 5 - Value of c is :" << c <<< endl ;
c = a++;
cout << "Line 6 - Value of c is :" << c <<< endl ;
c = a--;
```

```
    cout << "Line 7 - Value of c is :" << c <<< endl ;
    return 0;
```

When the above code is compiled and executed, it produces the following result:

Line 1 - Value of c is :31

Line 2 - Value of c is :11

Line 3 - Value of c is :210

Line 4 - Value of c is :2

Line 5 - Value of c is :1

Line 6 - Value of c is :21

Line 7 - Value of c is :22

Relational Operators

There are following relational operators supported by C++ language

Assume variable A holds 10 and variable B holds 20, then:

Operator Description Example

== Checks if the values of two operands (A == B) is not true.

are equal or not, if yes then

condition becomes true.

I= Checks if the values of two operands (A != B) is true.

are equal or not, if values are not

equal then condition becomes true.

> Checks if the value of left operand is (A > B) is not true.

greater than the value of right

operand, if yes then condition

becomes true.

< Checks if the value of left operandis (A < B) is true.
less than the value of right operand,
if yes then condition becomes true.

>= Checks if the value of left operand is (A >= B) is not true.
greater than or equal to the value of
right operand, if yes then condition
becomes true.

<= Checks if the value of left operand is (A <= B) is true.
less than or equal to the value of
right operand, if yes then condition
becomes true.

For example

```
#include <iostream>
using namespace std;
main()
{
int a = 21;
int b = 10;
int c;
if(a==b)
{
cout << "Line 1 - a is equal to b" <<< endl ;
else
{
```

```
cout << "Line 1 - a is not equal to b" <<< endl ;
if(a < b)
{
cout << "Line 2 - a is less than b" <<< endl ;

else
{
cout << "Line 2 - a is not less than b" <<< endl ;
}
if(a > b)
{
cout << "Line 3 - a is greater than b" <<< endl ;
}
else
{
cout << "Line 3 - a is not greater than b" <<< endl ;
}
/* Let's change the values of a and b */
a = 5;
b = 20;
```

Logical operators (!, &&, ||)

The operator ! is the C++ operator to perform the Boolean operation NOT. It has only one operand, located at its right, and it inverts its value, producing false if its operand is true and true if its operand is false. For example:

!(5 == 5) // evaluates to false because (5 == 5) is true.

!(6 <= 4) // evaluates to true because (6 <= 4) is false.

!true // evaluates to false
!false // evaluates to true

|| OPERATOR
.
For example:

((5 == 5) && (3 > 6)) evaluates to false (true && false).

((5 == 5) Il (3 > 6)) evaluates to true (true || false).

Conditional operator (?)

The conditional operator evaluates an expression, returning one value if that

expression is true and a different value if it is false. Its format is:

condition ? result1 : result2;

If condition is true, the expression returns result1; otherwise it returns result2.

```
#include <iostream>
using namespace std;
int main()
{
   int a = 2, b = 7, c;
   c = (a > b) ? a : b;
   cout << c;
   return 0;
}
```

In this example a was 2 and b was 7, so the expression (a > b) was false and the second value was selected.

Comma operator (,)

The comma operator (,) is used to separate two or more expressions that are included where only one expression
is expected. When the set of expressions has to be evaluated for a value, only the rightmost expression is
considered.

For example, the following code:

```
ja = (b=3, b+2);
```

Would first assign the value 3 to 2, and then assign +2 to variable 2. So, at the end, variable = would contain the

value 5 while variable » would contain value 3.

Bitwise Operators (&, |, ", ~, <<, >>)

Bitwise operators modify variables considering the bit patterns that represent the values they store.
loperatorjasm equivalent| description

[Q__AND J itwise AND

[for Bitwise Inclusive OR

Por Bitwise Exclusive OR.

[Not Unary complement (bit inversion)

[x fsHe Shift Left

b= |sHR_ (Shift Right

Explicit type casting operator
Type casting operators allow you to convert a datum of a given type to another. There are several ways to do this
in C++. The simplest one, which has been inherited from the C language, is to precede the expression to be
converted by the new type enclosed between parentheses (()):

```
float f = 3.14;

i = (int) f;
```

The previous code converts the float number 3.14 to an integer value (3), the remainder is lost. Here, the
typecasting operator was (int). Another way to do the same thing in C++ is using the functional notation:
preceding the expression to be converted by the type and enclosing the expression between parentheses:

```
i = int(f);
```

Both ways of type casting are valid in C++.
sizeof()
This operator accepts one parameter, which can be either a type or a variable itself and returns the size in bytes of
that type or object:

```
a = sizeof(char);
```

This will assign the value 1 to a because char is a one-byte long type.
The value returned by sizeof is a constant, so it is always determined before program execution.

4.1 Basic Input/Output

Until now, the example programs of previous sections provided very little interaction with the user, if any at all.
Using the standard input and output library, we will be able to interact with the user

by printing messages on the
screen and getting the user's input from the keyboard.
C++ uses a convenient abstraction called streams to perform input and output operations in sequential media such
as the screen or the keyboard. A stream is an object where a program can either insert or extract characters
to/from it. We do not really need to care about many specifications about the physical media associated with the
stream - we only need to know it will accept or provide characters sequentially.|
The standard C++ library includes the header file iostream, where the standard input and output stream objects
are declared.
Standard Output (cout)
By default, the standard output of a program is the screen, and the C++ stream object defined to access it is cout.
cout is used in conjunction with the insertion operator, which is written as << (two "less than" signs).

```
cout << "Output sentence"; // prints Output sentence on screen

cout << 120; // prints number 120 on screen

cout << x7 // prints the content of x on screen|
```

The << operator inserts the data that follows it into the stream preceding it. In the examples above it inserted the
constant string output sentence, the numerical constant 120 and variable x into the standard output stream cout.

Notice that the sentence in the first instruction is enclosed between double quotes (") because it is a constant
string of characters. Whenever we want to use constant strings of characters we must enclose them between
double quotes (") so that they can be clearly distinguished from variable names. For example, these two sentences
have very different results:

```
cout << "Hello"; // prints Hello

cout << Hello; // prints the content of Hello variable|
```

The insertion operator (<<) may be used more than once in a single statement:

```
cout << "Hello, " << "I am" << "a C++ statement";
```

This last statement would print the message Hello, I am a C++ statement on the screen. The utility of repeating
the insertion operator (<<) is demonstrated when we want to print out a combination of variables and constants or
more than one variable:

```
cout << "Hello, I am" << age << " years old and my zipcode is " << zipcode;
```

If we assume the age variable to contain the value 24 and the zipcode variable to contain 90064 the output of the
previous statement would be:

```
Hello, I am 24 years old and my zipcode is 90064
```

It is important to notice that cout does not add a line break after its output unless we

explicitly indicate it,
therefore, the following statements:

```
cout << "This is a sentence.";
cout << "This is another sentence.";
```

will be shown on the screen one following the other without any line break between them:

```
This is a sentence.This is another sentence.
```

even though we had written them in two different insertions into cout. In order to perform a line break on the
output we must explicitly insert a new-line character into cout. In C++ a new-line character can be specified as \n
(backslash, n):

```
cout << "First sentence.\n ";
cout << "Second sentence.\nThird sentence.";
```

This produces the following output:

```
First sentence.
Second sentence.
Third sentence.
```

Additionally, to add a new-line, you may also use the end1 manipulator. For example:

```
cout << "First sentence." <<< endl;
cout << "Second sentence." <<< endl;
```

would print out:

```
First sentence.
Second sentence.
```

The endl manipulator produces a newline character, exactly as the insertion of '\n' does, but it also has an
additional behavior when it is used with buffered streams: the buffer is flushed. Anyway, cout will be an
unbuffered stream in most cases, so you can generally use both the \n escape character and the endl manipulator
in order to specify a new line without any difference in its behavior.
Standard Input (cin).
The standard input device is usually the keyboard. Handling the standard input in C++ is done by applying the
overloaded operator of extraction (>>) on the cin stream. The operator must be followed by the variable that will
store the data that is going to be extracted from the stream. For example:

int age:

cin >> age;
The first statement declares a variable of type int called age, and the second one waits for an input from cin (the
keyboard) in order to store it in this integer variable.
cin can only process the input from the keyboard once the RETURN key has been pressed. Therefore, even if you
request a single character, the extraction from cin will not process the input until the user presses RETURN after
the character has been introduced.
You must always consider the type of the variable that you are using as a container with cin extractions. If you

request an integer you will get an integer, if you request a character you will get a character and if you request a
string of characters you will get a string of characters.

// i/o example Please enter an integer value: 702
The value you entered is 702 and its double is

#t#include <iostream> 1404.
using namespace std;
int main()
{

int i;

cout << "Please enter an integer value: ";

cin >> i

cout << "The value you entered is " << i7

cout << " and its double is " << i*2 << ".\n";

return 0;
}
The user of a program may be one of the factors that generate errors even in the simplest programs that use cin
(like the one we have just seen). Since if you request an integer value and the user

introduces a name (which
generally is a string of characters), the result may cause your program to misoperate since it is not what we were
expecting from the user. So when you use the data input provided by cin extractions you will have to trust that
the user of your program will be cooperative and that he/she will not introduce his/her name or something similar
when an integer value is requested. A little ahead, when we see the stringstream class we will see a possible
solution for the errors that can be caused by this type of user input.
You can also use cin to request more than one datum input from the user:

```
cin >> a >> b;
```

is equivalent to:

```
cin >> a;

cin >> b;
```

In both cases the user must give two data, one for variable a and another one for variable b that may be separated
by any valid blank separator: a space, a tab character or a newline.

cin and strings

We can use cin to get strings with the extraction operator (>>) as we do with fundamental data type variables:

```
cin >> mystring;
```

However, as it has been said, cin extraction stops reading as soon as if finds any blank space character, so in this

case we will be able to get just one word for each extraction. This behavior may or may not be what we want; for

example if we want to get a sentence from the user, this extraction operation would not be useful.

In order to get entire lines, we can use the function getline, which is the more recommendable way to get user

input with cin:

// cin with strings What's your name? Juan SouliA AzAx
#include <iostream> Hello Juan SouliA AzA%.
tinclude <string> What is your favorite team? The Isotopes
using namespace std; I like The Isotopes too!
int main()
{

string mystr;

cout << "What's your name? ";

getline (cin, mystr);

```
cout << "Hello " << mystr << ".\n";

cout << "What is your favorite team? ";

getline (cin, mystr);

cout << "I like " << mystr << " too!\n";

return 0;
}
```

Notice how in both calls to getline we used the same string identifier (mystr). What the program does in the
second call is simply to replace the previous content by the new one that is introduced.

Chapter 5: Control Structures

A program is usually not limited to a linear sequence of instructions. During its process it may bifurcate, repeat
code or take decisions. For that purpose, C++ provides control structures that serve to specify what has to be done
by our program, when and under which circumstances.

With the introduction of control structures we are going to have to introduce a new concept: the compound-

statement or block. A block is a group of statements which are separated by semicolons (;) like all C++
statements, but grouped together in a block enclosed in braces: { }:

{ statementl; statement2; statement3; }

Most of the control structures that we will see in this section require a generic statement as part of its syntax. A
statement can be either a simple statement (a simple instruction ending with a semicolon) or a compound
statement (several instructions grouped in a block), like the one just described. In the case that we want the
statement to be a simple statement, we do not need to enclose it in braces ({}). But in the case that we want the
statement to be a compound statement it must be enclosed between braces ({}), forming a block.

Conditional structure: if and else

The if keyword is used to execute a statement or block only if a condition is fulfilled. Its form is:

if (condition) statement

Where condition is the expression that is being evaluated. If this condition is true, statement is executed. If it is
false, statement is ignored (not executed) and the program continues right after this conditional structure.

For example, the following code fragment prints x is 100 only if the value stored in the x variable is indeed 100:

```
if (x == 100)
cout << "x is 100";
```

If we want more than a single statement to be executed in case that the condition is true we can specify a block using braces { }:

```
if (x == 100)
{
cout << "x is ";
cout << x;
}
```

We can additionally specify what we want to happen if the condition is not fulfilled by using the keyword else. Its form used in conjunction with if is:

if (condition) statementl else statement2

For example:

```
if (x == 100)
cout << "x is 100";
else
cout << "x is not 100";
```

prints on the screen x is 100 if indeed x has a value of 100, but if it has not -and only if not- it prints out x is not 100.

The if + else structures can be concatenated with the intention of verifying a range of values. The following example shows its use telling if the value currently stored in x is positive, negative or none of them (i.e. zero):

```
if (x > 0)
cout << "x is positive";
else if (x < 0)
cout << "x is negative";
else
cout << "x is 0";
```

Remember that in case that we want more than a single statement to be executed, we must group them in a block
by enclosing them in braces { }.

Iteration structures (loops)

Loops have as purpose to repeat a statement a certain number of times or while a condition is fulfilled.

The while loop

Its format is:

while (expression) statement

and its functionality is simply to repeat statement while the condition set in expression is true.

For example, we are going to make a program to countdown using a while-loop:

```
77 custom countdown using while Enter the starting number > 8
8, 7, 6, 5, 4, 3, 2, 1, FIRE!
```

```
#include <iostream>
using namespace std;
int main()
{
```

```
int n;

cout << "Enter the starting number > ";

cin >> nj;

while (n>0) {

cout << n <<", "|
--n;

}

cout << "FIRE! \n";

return 0;
}
```

When the program starts the user is prompted to insert a starting number for the countdown. Then the while loop
begins, if the value entered by the user fulfills the condition n> (that n is greater than zero) the block that follows
the condition will be executed and repeated while the condition (n>0) remains being true.
The whole process of the previous program can be interpreted according to the following script (beginning in
main):

1. User assigns a value to n

2. The while condition is checked (n>0). At this point there are two posibilities:

* condition is true: statement is executed (to step 3)

* condition is false: ignore statement and continue after it (to step 5)

3. Execute statement:

cout << n <<", 0";

--nj

(prints the value of n on the screen and decreases n by 1)

4. End of block. Return automatically to step 2

5. Continue the program right after the block: print FIRE! and end program.

When creating a while-loop, we must always consider that it has to end at some point, therefore we must provide

within the block some method to force the condition to become false at some point, otherwise the loop will

continue looping forever. In this case we have included --n; that decreases the value of the variable that is being

evaluated in the condition (n) by one - this will eventually make the condition (n>0) to become false after a certain

number of loop iterations: to be more specific, when n becomes 0, that is where our while-loop and our countdown

end.

Of course this is such a simple action for our computer that the whole countdown is performed instantly without

any practical delay between numbers.

The do-while loop

Its format is:

do statement while (condition);

Its functionality is exactly the same as the while loop, except that condition in the do-while loop is evaluated after
the execution of statement instead of before, granting at least one execution of statement even if condition is
never fulfilled. For example, the following example program echoes any number you enter until you enter 0.

```
// number echoer
#include <iostream>
using namespace std;
int main()
{
  unsigned long n;
  do {
    cout << "Enter number (0 to end): ";
    cin >> n;
    cout << "You entered: " << n << "\n";
  } while (n != 0);
  return 0;
}
```

Screen output:

```
Enter number (0 to end): 12345
You entered: 12345
Enter number (0 to end): 160277
You entered: 160277
Enter number (0 to end): 0
```

You entered: 0

The do-while loop is usually used when the condition that determines the end of the loop is determined within the loop statement itself.

The for loop

Its format is:

for (initialization; condition; increase) statement;

and its main function is to repeat statement while condition remains true, like the while loop. But in addition, the
for loop provides specific locations to contain an initialization statement and an increase statement. So this
loop is specially designed to perform a repetitive action with a counter which is initialized and increased on each
iteration.

It works in the following way:

1. initialization is executed. Generally it is an initial value setting for a counter variable. This is executed
only once.
2. condition is checked. If it is true the loop continues, otherwise the loop ends and statement is skipped
(not executed).
3. statement is executed. As usual, it can be either a single statement or a block enclosed in braces { }.
4. finally, whatever is specified in the increase field is executed and the loop gets back to step 2.

Here is an example of countdown using a for loop:

77 countdown using a for loop fo, 9, 8, 7, 6, 5, 4, 3, 2, 1, FIRE!

```
#include <iostream>
using namespace std;
int main()
{
for (int n=10; n>0; n--) {
cout << n <<", "|
}
cout << "FIRE! \n";
return 0;
}
```

The initialization and increase fields are optional. They can remain empty, but in all cases the semicolon signs
between them must be written. For example we could write: for (;n<10;) if we wanted to specify no initialization
and no increase; or for (;n<10;n++) if we wanted to include an increase field but no initialization (maybe because
the variable was already initialized before).

Optionally, using the comma operator (,) we can specify more than one expression in any of the fields included in
a Zox loop, like in inicialization, for example. The comma operator (,) is an expression separator, it serves to
separate more than one expression where only one is generally expected. For example, suppose that we wanted to
initialize more than one variable in our loop:

```
for ( n=0, i=100 ; n!=i ; n++, i-- )
```

{
whatever here...
A
This loop will execute for 50 times if neither n or i are modified within the loop:
Initialize tiog
Comdi tion
for {20s @=i00]: 3 =
Inerease
2 starts with a value of 0, and 4 with 109, the condition is n!=4 (that n is not equal to).
Because » is increased by
one and i decreased by one, the loop's condition will become false after the 50th loop,
when both = and i will be
equal to 50.

Jump statements.
The break statement
Using 2xeak we can leave a loop even if the condition for its end is not fulfilled. It can be used to end an infinite
loop, or to force it to end before its natural end. For example, we are going to stop the count down before its
natural end (maybe because of an engine check failure?):
break loop example id, 5, 8, 7, 6, 5, 4, 3, countdown aborted!
#include <iostream>
using namespace std;
int main ()
{
int nj

```
for (n=10; n>0; n--)
cout <<n <<", "|
if (n==3)
{
break;
}
return 0;
```

I

The continue statement

The continue statement causes the program to skip the rest of the loop in the current iteration as if the end of the

statement block had been reached, causing it to jump to the start of the following iteration. For example, we are

going to skip the number 5 in our countdown:

7 continue loop example fio, 9, 8, 7, 6, 4, 3, 2, 1, FIRB!

```
#include <iostream>
using namespace std;
int main()
{
for (int n=10; n>0; n--) {
if (n==5) continue;
cout <on cc", "|
}
cout << "FIRE! \n";
return 0;
}
```

The selective structure: switch.

The syntax of the switch statement is a bit peculiar. Its objective is to check several possible constant values for an

expression. Something similar to what we did at the beginning of this section with the concatenation of several if

and else if instructions. Its form is the following:

```
switch (expression)
{
case constantl:
group of statements 1;
break;
case constant2:
group of statements 2;
break;
default:
default group of statements
}
```

It works in the following way: switch evaluates expression and checks if it is equivalent to constant1, if itis, it

executes group of statements 1 until it finds the break statement. When it finds this break statement the

program jumps to the end of the switch selective structure.

If expression was not equal to constant it will be checked against constant2. If it is equal to this, it will execute

group of statements 2 until a break keyword is found, and then will jump to the end of the switch selective

structure.

Finally, if the value of expression did not match any of the previously specified constants (you can include as
many case labels as values you want to check), the program will execute the statements included after the
default: label, if it exists (since it is optional).

Both of the following code fragments have the same behavior:

switch example if-else equivalent

pescch (=) if (x == 1) {

cece << Me is 1": cout << "x is 1";

c < sl"; }

anew else if (x == 2) {

vemout << "% is 2"; cout << "x is 2"

break; eve

default: else { " . "

—— cout << "value of x unknown";

cout << "value of x unknown"s|

}

The switch statement is a bit peculiar within the C++ language because it uses labels instead of blocks. This
forces us to put break statements after the group of statements that we want to be executed for a specific
condition. Otherwise the remainder statements -including those corresponding to other labels- will also be
executed until the end of the switch selective block or a break statement is reached.

For example, if we did not include a break statement after the first group for case one,

the program will not
automatically jump to the end of the switch selective block and it would continue executing the rest of statements
until it reaches either a break instruction or the end of the switch selective block. This makes unnecessary to
include braces { } surrounding the statements for each of the cases, and it can also be useful to execute the same
block of instructions for different possible values for the expression being evaluated.
For example:

```
switch (x) {
case 1:
case 2:
case 3:
cout << "x is 1, 2 or 3"
break;
default:
cout << "x is not 1, 2 nor 3"%
}
```

5.1 C++ Exceptions

When executing C++ code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other
unforeseeable things.
When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an exception
(throw an error).
C++ try and catch

Exception handling in C++ consist of three keywords: try, throw and catch:

The try statement allows you to define a block of code to be tested for errors while it is being executed.

The throw keyword throws an exception when a problem is detected, which lets us create a custom error.

The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.

The try and catch keywords come in pairs:

Example

```
try {
// Block of code to try
throw exception; // Throw an exception when a problem arise
}
catch () {
// Block of code to handle errors
}
```

Consider the following example:

Example

```
try {
int age = 15;
if (age >= 18) {
cout << "Access granted - you are old enough.";
} else {
throw (age);
}
}
```

```
  catch (int myNum) {
  cout << "Access denied - You must be at least 18 years old.\n";
  cout << "Age is: " << myNum;
  }
```

Output:

```
Access denied - You must be at least 18 years old.
Age is: 15
```

Example explained

We use the try block to test some code: If the age variable is less than 18, we will throw an exception, and handle it in our catch
block.

In the catch block, we catch the error and do something about it. The catch statement takes a parameter: in our example we use an
int variable (myNum) (because we are throwing an exception of int type in the try block (age)), to output the value of age .

If no error occurs (e.g. if age is 20 instead of 15, meaning it will be be greater than 18), the catch block is skipped:

Example

```
int age = 20;
```

You can also use the throw keyword to output a reference number, like a custom error number/code for organizing purposes (505 in our example):

Example

```
try {
int age = 15;
if (age >= 18) {
cout << "Access granted - you are old enough.";
} else {
throw 505;
}
}
catch (int myNum) {
cout << "Access denied - You must be at least 18 years old.\n";
cout << "Error number: " << myNum;
}
```

Output:

```
Access denied - You must be at least 18 years old.
Error number: 55
```

Handle Any Type of Exceptions (...)

If you do not know the throw type used in the try block, you can use the "three dots" syntax (...) inside the catch block, which will handle any type of exception:

Example

```
try {
int age = 15;
```

```
  if (age >= 18) {
  cout << "Access granted - you are old enough.";
  } else {
  throw 505;
  }
  }
  catch (...) {
  cout << "Access denied - You must be at least 18 years old.\n";
  }
```

Output:

Access denied - You must be at least 18 years old.

Chapter 6: Functions

Using functions we can structure our programs in a more modular way, accessing all the potential that structured
programming can offer to us in C++.
A function is a group of statements that is executed when it is called from some point of the program. The
following is its format:
type name (parameterl, parameter2, ...) { statements }
where:
@ type is the data type specifier of the data returned by the function.
e@ name is the identifier by which it will be possible to call the function.
@ parameters (as many as needed): Each parameter consists of a data type specifier followed by an
identifier, like any regular variable declaration (for example: int x) and which acts within the function as
a regular local variable. They allow to pass arguments to the function when it is called. The different
parameters are separated by commas.
@ statements is the function's body. It is a block of statements surrounded by braces { }.
Here you have the first function example:

```
// function example
#include <iostream>
using namespace std;
```

```
int addition(int a, int b)
{
   int r;
   r = a + b;
   return r;
}
int main()
{
   int z;
   z = addition(5, 3);
   cout << "The result is " << z;
   return 0;
}
```

Screen output:
The result is 8

In order to examine this code, remember that a C++ program begins its execution with the main function. The parameters and arguments have a clear correspondence: addition(5, 3) passes 5 and 3 to parameters a and b.

At the point at which the function is called from within main, the control is lost by main and passed to function
addition. The value of both arguments passed in the call (5 and 3) are copied to the local variables int a and int
b within the function.
Function addition declares another local variable (int r), and by means of the

expression r=a+b, it assigns to r
the result of a plus 5. Because the actual parameters passed for a and » are 5 and 3 respectively, the result is 8.
The following line of code:
return (r)7

finalizes function addition, and returns the control back to the function that called it in the first place (in this case,
main). At this moment the program follows it regular course from the same point at which it was interrupted by the
call to addition. But additionally, because the return statement in function addition specified a value: the
content of variable r (return (r);), which at that moment had a value of s. This value becomes the value of
evaluating the function call.

int addition (int a, int b)

ag

z= addition { 5 , re
So being the value returned by a function the value given to the function call itself when it is evaluated, the
variable z will be set to the value returned by addition (5, 3), that is 8. To explain it another way, you can
imagine that the call to a function (addition (5, 3)) is literally replaced by the value it returns (8).

The following line of code in main is:

```
cout << "The result is " << z;
```

That, as you may already expect, produces the printing of the result on the screen.

Another example:

```
// function example
#include <iostream>
using namespace std;
int subtraction(int a, int b)
{
   int r;
   r = a - b;
   return r;
}
int main()
{
   int x = 5, y = 3, z;
   z = subtraction(7, 2);
   cout << "The first result is " << z << '\n';
   cout << "The second result is " << subtraction(7, 2) << '\n';
   cout << "The third result is " << subtraction(x, y) << '\n';
   z = 4 + subtraction(x, y);
   cout << "The fourth result is " << z << '\n';
   return 0;
}
```

Screen output:

The first result is 5

The second result is 5

The third result is 2

The fourth result is 6

In this case we have created a function called subtraction. The function subtracts the two passed parameters and returns the result.

If we replace the function call by the value it returns (i.e., 5), we would have:

z2= 57

cout << "The first result is " << z7

As well as

cout << "The second result is " << subtraction (7,2);

has the same result as the previous call, but in this case we made the call to subtraction directly as an insertion

parameter for cout. Simply consider that the result is the same as if we had written:

cout << "The second result is " << 5;

since 5 is the value returned by subtraction (7,2).

In the case of:

cout << "The third result is " << subtraction (x,y)

The only new thing that we introduced is that the parameters of subtraction are variables instead of constants.

That is perfectly valid. In this case the values passed to function subtraction are the values of x and y, that are 5

and 3 respectively, giving 2 as result.
The fourth case is more of the same. Simply note that instead of:

z = 4 + subtraction (x,y)?
we could have written:

z = subtraction (x,y) + 4;
with exactly the same result. I have switched places so you can see that the semicolon sign (;) goes at the end of
the whole statement. It does not necessarily have to go right after the function call. The explanation might be once
again that you imagine that a function can be replaced by its returned value:

z=4+2;

z=24+ 4;

Functions with no type. The use of void.
If you remember the syntax of a function declaration:
type name (argumentl, argument2 ...) statement
you will see that the declaration begins with a type, that is the type of the function itself (i.e., the type of the
datum that will be returned by the function with the return statement). But what if we want to return no value?
Imagine that we want to make a function just to show a message on the screen. We do not need it to return any
value. In this case we should use the void type specifier for the function. This is a

special specifier that indicates
absence of type.

```
// void function example I'm a function!
#include <iostream>
using namespace std;
oid printmessage ()
{

cout << "I'm a function!";
}
int main()
{

printmessage ()7

return 0;
}
```

void can also be used in the function's parameter list to explicitly specify that we want the function to take no
actual parameters when it is called. For example, function printmessage could have been declared as:

```
void printmessage (void)

{
cout << "I'm a function!";
}
```

Although it is optional to specify void in the parameter list. In C++, a parameter list can simply be left blank if we
want a function with no parameters.

What you must always remember is that the format for calling a function includes specifying its name and
enclosing its parameters between parentheses. The non-existence of parameters does not exempt us from the
obligation to write the parentheses. For that reason the call to printmessage is:

printmessage ()7

[redacted]

Chapter 7: ARRAYS

An array is a series of elements of the same type placed in contiguous memory locations that can be individually
referenced by adding an index to a unique identifier.
That means that, for example, we can store 5 values of type int in an array without having to declare 5 different
variables, each one with a different identifier. Instead of that, using an array we can store 5 different values of the
same type, int for example, with a unique identifier.
For example, an array to contain 5 integer values of type int called billy could be represented like this:

0 1 2 3 4

bly TT

__ __ _____

int

where each blank panel represents an element of the array, that in this case are integer values of type int. These

elements are numbered from 0 to 4 since in arrays the first index is always 0, independently of its length.

Like a regular variable, an array must be declared before it is used. A typical declaration for an array in C++ is:

type name [elements];

where type is a valid type (like int, float...), name is a valid identifier and the elements field (which is always

enclosed in square brackets []), specifies how many of these elements the array has to contain.

Therefore, in order to declare an array called billy as the one shown in the above diagram it is as simple as:

int billy [5];

NOTE: The elements field within brackets [] which represents the number of elements the array is going to hold,

must be a constant value, since arrays are blocks of non-dynamic memory whose size must be determined before

execution. In order to create arrays with a variable length dynamic memory is needed, which is explained later in

these tutorials.

Initializing arrays.

When declaring a regular array of local scope (within a function, for example), if we do not specify otherwise, its
elements will not be initialized to any value by default, so their content will be undetermined until we store some
value in them. The elements of global and static arrays, on the other hand, are automatically initialized with their
default values, which for all fundamental types this means they are filled with zeros.
In both cases, local and global, when we declare an array, we have the possibility to assign initial values to each
one of its elements by enclosing the values in braces { }. For example:

int billy [5] = { 16, 2, 77, 40, 12071 }+
This declaration would have created an array like this:

o 1 2 3 4
bly. 2] SO]
The amount of values between braces { } must not be larger than the number of elements that we declare for the
array between square brackets [ ]. For example, in the example of array billy we have declared that it has 5
elements and in the list of initial values within braces { } we have specified 5 values, one for each element.
When an initialization of values is provided for an array, C++ allows the possibility of leaving the square brackets
empty [1]. In this case, the compiler will assume a size for the array that matches the number of values included
between braces { }:

int billy [] = { 16, 2, 77, 40, 12071 };

After this declaration, array billy would be 5 ints long, since we have provided 5 initialization values.

Accessing the values of an array.

In any point of a program in which an array is visible, we can access the value of any of its elements individually as

if it was a normal variable, thus being able to both read and modify its value. The format is as simple as:

name [index]

Following the previous examples in which billy had 5 elements and each of those elements was of type int, the

name which we can use to refer to each element is the following:

billy[0] billyf1] billy[2] billy[3] billy[4]

bily(

For example, to store the value 75 in the third element of billy, we could write the following statement:

Ipilly[2] = 75;

and, for example, to pass the value of the third element of billy to a variable called a, we could write:

a = billy[2];

Therefore, the expression billy(2] is for all purposes like a variable of type int.

Notice that the third element of billy is specified billy[2], since the first one is billy[0], the second one is

billy[1], and therefore, the third one is billy[2]. By this same reason, its last element is billy [4]. Therefore, if

we write billy[5], we would be accessing the sixth element of billy and therefore exceeding the size of the array.

In C++ it is syntactically correct to exceed the valid range of indices for an array. This can create problems, since accessing out-of-range elements do not cause compilation errors but can cause runtime errors. The reason why this is allowed will be seen further ahead when we begin to use pointers.

At this point it is important to be able to clearly distinguish between the two uses that brackets [ ] have related to arrays. They perform two different tasks: one is to specify the size of arrays when they are declared; and the second one is to specify indices for concrete array elements. Do not confuse these two possible uses of brackets [ ] with arrays.

```
int billy[5]; // declaration of a new array
billy[2] = 75; // access to an element of the array.
```

If you read carefully, you will see that a type specifier always precedes a variable or array declaration, while it never precedes an access.

Some other valid operations with arrays:

```
billy[0] = a;
billy[a] = 75;
b = billy[a+2];
billy[billy[a]] = billy[2] + 5;
```

```
// arrays example
#include <iostream>
```

```
using namespace std;
int billy[] = {16, 2, 77, 40, 12071};
int n, result = 0;
int main()
{
  for (n = 0; n < 5; n++)
    result += billy[n];
  cout << result;
  return 0;
}
```

o 1 2 3 4

LI}

jimmy 1

2

jimmy represents a bidimensional array of 3 per 5 elements of type int. The way to declare this array in C++

would be:

int jimmy [3] [5];

and, for example, the way to reference the second element vertically and fourth horizontally in an expression

would be:

jimmy [1] [3]

o 1 2 a 4

- o

Jimny 1

2

jimmy (11 [3]

(remember that array indices always begin by zero).

Multidimensional arrays are not limited to two indices (i.e., two dimensions). They can contain as many indices as

needed. But be careful! The amount of memory needed for an array rapidly increases with each dimension. For

example:

char century [100] [365] [24] [60] [60];

declares an array with a char element for each second in a century, that is more than 3 billion chars. So this

declaration would consume more than 3 gigabytes of memory!

Multidimensional arrays are just an abstraction for programmers, since we can obtain the same results with a

simple array just by putting a factor between its indices:

int jimmy [3][5]; // is equivalent to

int jimmy [15]; // (3 * 5 = 15)

With the only difference that with multidimensional arrays the compiler remembers the depth of each imaginary

dimension for us. Take as example these two pieces of code, with both exactly the same result. One uses a

bidimensional array and the other one uses a simple array:

multidimensional array pseudo-multidimensional array

#define WIDTH 5 #define WIDTH 5

define HEIGHT 3 define HEIGHT 3

```
int jimmy [HEIGHT] [WIDTH]; int jimmy [HEIGHT * WIDTH];
int n,m int n,m;
int main () int main ()
{ {

for (n=0;n<HEIGHT;n++) for (n=0;n<HEIGHT;n++)
for (m=0;m<WIDTH;m++) for (m=0;m<WIDTH;m++)
{ {
jimmy [n] [m]=(n+1) * (m+1)7 jimmy [n*WIDTH+m]=(n+1)* (m+1);
} }
return 0; return 0;
} }
```

None of the two source codes above produce any output on the screen, but both assign values to the memory
block called jimmy in the following way:

```
o 1 2 3 4
jimmy} SE 2 [ 2 [ 3 Ta Ts
}
a
```

We have used "defined constants" (#define) to simplify possible future modifications of the program. For example,
in case that we decided to enlarge the array to a height of 4 instead of 3 it could be done simply by changing the
line:

```
#define HEIGHT 3
```

to:

#define HEIGHT 4

with no need to make any other modifications to the program.

Arrays as parameters

At some moment we may need to pass an array to a function as a parameter. In C++ it is not possible to pass a complete block of memory by value as a parameter to a function, but we are allowed to pass its address. In practice this has almost the same effect and it is a much faster and more efficient operation.

In order to accept arrays as parameters the only thing that we have to do when declaring the function is to specify in its parameters the element type of the array, an identifier and a pair of void brackets []. For example, the following function:

void procedure (int arg[])

accepts a parameter of type "array of int" called arg. In order to pass to this function an array declared as:

int myarray [40];

it would be enough to write a call like this:

procedure(myarray);

Here you have a complete example:

// arrays as parameters

```
#include <iostream>
using namespace std;
void printarray(int arg[], int length)
{
   for (int n = 0; n < length; n++)
      cout << arg[n] << " ";
   cout << "\n";
}
int main()
{
   int firstarray[] = {5, 10, 15};
   int secondarray[] = {2, 4, 6, 8, 10};
   printarray(firstarray, 3);
   printarray(secondarray, 5);
   return 0;
}
```

As you can see, the first parameter (int arg[]) accepts any array whose elements are of type int.

As you may already know, the C++ Standard Library implements a powerful string class, which is very useful to handle and manipulate strings of characters. However, because strings are in fact sequences of characters, we can represent them also as plain arrays of char elements.

For example, the following array:

char jenny [20];

is an array that can store up to 20 elements of type char. It can be represented as:

jenny

Therefore, in this array, in theory, we can store sequences of characters up to 20 characters long. But we can also
store shorter sequences. For example, jenny could store at some point in a program either the sequence "Hello"
or the sequence "Merry christmas", since both are shorter than 20 characters.
Therefore, since the array of characters can store shorter sequences than its total length, a special character is
used to signal the end of the valid sequence: the null character, whose literal constant can be written as '\0'
(backslash, zero).
Our array of 20 elements of type char, called jenny, can be represented storing the characters sequences "Hello"
and "Merry Christmas" as:

jenny

fate [1 [a Jo fio]

Pefefe|e|y] Jefe] |i]s]t[=]a]s fro)

Notice how after the valid content a null character ('\0') has been included in order to indicate the end of the
sequence. The panels in gray color represent char elements with undetermined values.
Initialization of null-terminated character sequences
Because arrays of characters are ordinary arrays they follow all their same rules. For

example, if we want to
initialize an array of characters with some predetermined sequence of characters we can do it just like any other
array:

char myword[] = { 'H', 'e', '1', '1', 'o', '\O" }e
In this case we would have declared an array of 6 elements of type char initialized with the characters that form
the word "Hello" plus a null character '\o' at the end.
But arrays of char elements have an additional method to initialize their values: using string literals.
In the expressions we have used in some examples in previous chapters, constants that represent entire strings of
characters have already showed up several times. These are specified enclosing the text to become a string literal
between double quotes ("). For example:

"the result is: "

is a constant string literal that we have probably used already.
Double quoted strings (") are literal constants whose type is in fact a null-terminated array of characters. So string
literals enclosed between double quotes always have a null character ('\0') automatically appended at the end.
Therefore we can initialize the array of char elements called myword with a null-terminated sequence of characters
by either one of these two methods:

char myword [] = { 'H', 'e', '1l', '1', 'o', "\O" }e

char myword [] = "Hello";
In both cases the array of characters myword is declared with a size of 6 elements of type char: the 5 characters
that compose the word "Hello" plus a final null character ('\0') which specifies the end of the sequence and that,
in the second case, when using double quotes (") it is appended automatically.
Please notice that we are talking about initializing an array of characters in the moment it is being declared, and
not about assigning values to them once they have already been declared. In fact because this type of null-
terminated arrays of characters are regular arrays we have the same restrictions that we have with any other
array, SO we are not able to copy blocks of data with an assignment operation.
Assuming mystext is a char[] variable, expressions within a source code like:

mystext = "Hello";

Imystext[] = "Hello";
would not be valid, like neither would be:

Imystext = { 'H', 'e', '1', '1', 'o', '\O" }i
The reason for this may become more comprehensible once you know a bit more about pointers, since then it will
be clarified that an array is in fact a constant pointer pointing to a block of memory.

Chapter 8: FILE I/O

8.1 C++ Files

The fstream library allows us to work with files.

To use the fstream library, include both the standard <iostream> AND the <fstream> header file:

Example

#include <iostream>

#include <fstream>

There are three classes included in the fstream library, which are used to create, write or read files:

Class Description

ofstream Creates and writes to files

ifstream Reads from files

fstream A combination of ofstream and ifstream: creates, reads, and writes to files

Create and Write To a File

To create a file, use either the ofstream or fstream class, and specify the name of the file.

To write to the file, use the insertion operator (<<).

Example

#include <iostream>

#include <fstream>

using namespace std;

```
int main() {
// Create and open a text file
ofstream MyFile("filename.txt");
// Write to the file
MyFile << "Files can be tricky, but it is fun enough!";
// Close the file
MyFile.close();
}
```

Why do we close the file?

It is considered good practice, and it can clean up unnecessary memory space.

Output :

Files can be tricky, but it is fun enough!

Read a File

To read from a file, use either the ifstream or fstream class, and the name of the file.

Note that we also use a while loop together with the getline() function (which belongs to the ifstream class) to read the file line by

line, and to print the content of the file:

Example

```
// Create a text string, which is used to output the text file
string myText;
// Read from the text file
ifstream MyReadFile("filename.txt");
// Use a while loop together with the getline() function to read the file line by line
while (getline (MyReadFile, myText)) {
// Output the text from the file
cout << myText;
```

```
}
// Close the file
MyReadFile.close();
```

Output:

Files can be tricky, but it is fun enough!

Chapter 9: Object Oriented Programming

OOP stands for Object-Oriented Programming. Procedural programming is about writing procedures or functions that perform operations on the data, while object-oriented programming is about creating objects that contain both data and functions. Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

C++ What are Classes and Objects?

Classes and objects are the two main aspects of object-oriented programming.

Look at the following illustration to see the difference between class and objects:

class objects

Fruit Apple

Banana

Mango

So, a class is a template for objects, and an object is an instance of a class. When the individual objects are created, they inherit all the variables and functions from the class.

9.1 C++ Classes and Objects

C++ is an object-oriented programming language. Everything in C++ is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an **object**. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

Attributes and methods are basically **variables** and **functions** that belongs to the class. These are often referred to as "class members".

A class is a user-defined data type that we can use in our program, and it works as an object constructor, or a "blueprint" for creating objects. To create a class, use the class keyword:

Example

Create a class called "MyClass ":

```
class MyClass { // The class
public: // Access specifier
int myNum; // Attribute (int variable)
string myString; // Attribute (string variable)
aa)
```

Example explained

* The class keyword is used to create a class called MyClass.

* The public keyword is an access specifier, which specifies that members (attributes and methods) of the class are
accessible from outside the class. You will learn more about access specifiers later.
* Inside the class, there is an integer variable myNum and a string variable myString . When variables are declared within a
class, they are called attributes.
« At last, end the class definition with a semicolon ; .

Create an Object
In C++, an object is created from a class. We have already created the class named MyClass , so now we can use this to create
objects.
To create an object of MyClass , specify the class name, followed by the object name.
To access the class attributes (myNum and myString), use the dot syntax (.) on the object:
Example
Create an object called " myObj " and access the attributes:

```
class MyClass { // The class
public: // Access specifier
int myNum; // Attribute (int variable)
string myString; // Attribute (string variable)
aa)
int main() {
MyClass myObj; // Create an object of MyClass
// Access attributes and set values
myObj.myNum = 15;
myObj.myString = "Some text";
```

```
// Print attribute values
cout << myObj.myNum << "\n";
cout << myObj.myString;
return 0;
}
```

Output:

```
15
Some text
```

Multiple Objects

You can create multiple objects of one class:

Example

```
// Create a Car class with some attributes
class Car {
public:
string brand;
string model;
int year;
}
int main() {
// Create an object of Car
Car carObj1;
carObj1.brand = "BMW";
carObj1.model = "X5";
carObj1.year = 1999;
// Create another object of Car
Car carObj2;
```

```
carObj2.brand = "Ford";
carObj2.model = "Mustang";
carObj2.year = 1969;
// Print attribute values
cout << carObj1.brand <<" " << carObj1.model <<" " << carObj1.year << "\n";

cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
return 0;
}
```

Inside Example

```
class MyClass { // The class
public: // Access specifier
void myMethod() { // Method/function defined inside the class
cout << "Hello World!";
}
}
int main() {
MyClass myObj; // Create an object of MyClass
myObj.myMethod(); // Call the method
return 0;
}
```

Output:

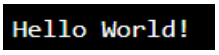
```
Hello World!
```

To define a function outside the class definition, you have to declare it inside the class and then define it outside of the class. This is done by
specifying the name of the class, followed the scope resolution :: operator, followed

by the name of the function:

Outside Example

```
class MyClass { // The class
public: // Access specifier
void myMethod();  // Method/function declaration
}
// Method/function definition outside the class
void MyClass::myMethod() {
cout << "Hello World!";
}
int main() {
MyClass myObj; // Create an object of MyClass
myObj.myMethod(); // Call the method
return 0;
}
```

Output: 

Parameters

You can also add parameters:

Example

```
#include <iostream>
using namespace std;
class Car {
public:
int speed(int maxSpeed) ;
}
```

```
int Car::speed(int maxSpeed) {
return maxSpeed;
}
int main() {
Car myObj; // Create an object of Car
cout << myObj.speed(200); // Call the method with an argument
return 0;
}
```

Output: 200

9.2 Constructor

Constructors

A constructor in C++ is a special method that is automatically called when an object of a class is created.

To create a constructor, use the same name as the class, followed by parentheses () :

Example

```
class MyClass { // The class
public: // Access specifier
MyClass() { // Constructor
cout << "Hello World!";
}
}
int main() {
MyClass myObj; // Create an object of MyClass (this will call the constructor)
```

```
return 0;
}
```

Output: `Hello World!`

Constructor Parameters

Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

The following class have brand, model and year attributes, and a constructor with different parameters. Inside the constructor we set the

attributes equal to the constructor parameters (brand=x , etc). When we call the constructor (by creating an object of the class), we pass

parameters to the constructor, which will set the value of the corresponding attributes to the same:

Example

```
class Car { // The class
public: // Access specifier
string brand; // Attribute
string model; // Attribute
int year; // Attribute
Car(string x, string y, int z) { // Constructor with parameters
brand = x;
model = y;
}
35
int main() {
// Create Car objects and call the constructor with different values
```

```
Car carObj1("BMW", "X5", 1999);
Car carObj2("Ford", "Mustang", 1969);
// Print values
cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";
cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
return 0;
}
```

Output:

```
BMW X5 1999
Ford Mustang 1969
```

Just like functions, constructors can also be defined outside the class. First, declare the constructor inside the class, and then define it outside of the class by specifying the name of the class, followed by the scope resolution :: operator, followed by the name of the constructor (which is the same as the class):

For example:

Just like functions, constructors can also be defined outside the class. First, declare the constructor inside the class, and then define it outside of the class by specifying the name of the class, followed by the scope resolution :: operator, followed by the name of the constructor (which is the same as the class):

Example

```
class Car { // The class
public: // Access specifier
string brand; // Attribute
string model; // Attribute
```

```
int year; // Attribute
Car(string x, string y, int z); // Constructor declaration
}
// Constructor definition outside the class
Car::Car(string x, string y, int z) {
brand = x;
model = y;
year = z;
}
int main() {
// Create Car objects and call the constructor with different values
Car carObj1("BMW", "X5", 1999);
Car carObj2("Ford", "Mustang", 1969);
// Print values
cout << carObj1.brand <<" "" << carObj1.model <<" " << carObj1-year << "\n";
cout << carObj2.brand <<" " << carObj2.model <<" " << carObj2.year << "\n";
return 0;
```

Output:

```
BMW X5 1999
Ford Mustang 1969
```

Access Specifiers

By now, you are quite familiar with the public keyword that appears in all of our class examples:

Example

```
class MyClass { // The class
public: // Access specifier
// class members goes here
```

}

Try it Yourself »

The public keyword is an access specifier. Access specifiers define how the members (attributes and methods) of a class can be
accessed. In the example above, the members are public - which means that they can be accessed and modified from outside the code.

However, what if we want members to be private and hidden from the outside world?

In C++, there are three access specifiers:

* public - members are accessible from outside the class

« private - members cannot be accessed (or viewed) from outside the class

« protected - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. You will learn
more about Inheritance later.

In the following example, we demonstrate the differences between public and private members:

Example

```
class MyClass {
public: // Public access specifier
int x; // Public attribute
private: // Private access specifier
int y;_ // Private attribute
}
int main() {
MyClass myObj;
myObj.x = 25; // Allowed (public)
myObj-y = 50; // Not allowed (private)
```

return 0;

}

If you try to access a private member, an error occurs:

Note: It is possible to access private members of a class using a public method inside the same class. See the next chapter (Encapsulation)
on how to do this.

Tip: It is considered good practice to declare your class attributes as private (as often as you can). This will reduce the possibility of yourself
(or others) to mess up the code. This is also the main ingredient of the Encapsulation concept, which you will learn more about in the next
chapter.

Note: By default, all members of a class are private if you don't specify an access specifier:

Example

```
class MyClass {
int x; // Private attribute
int y; // Private attribute
}
```

9.3 Encapsulation

The meaning of Encapsulation, is to make sure that "sensitive" data is hidden from users. To achieve this, you must declare class
variables/attributes as private (cannot be accessed from outside the class). If you want others to read or modify the value of a private
member, you can provide public get and set methods.

Encapsulation

The meaning of Encapsulation, is to make sure that "sensitive" data is hidden from users. To achieve this, you must declare class variables/attributes as private (cannot be accessed from outside the class). If you want others to read or modify the value of a private member, you can provide public get and set methods.

Access Private Members

To access a private attribute, use public "get" and "set" methods:

Example

```
#include <iostream>
using namespace std;
class Employee {
private:
// Private attribute
int salary;
public:
// Setter
void setSalary(int s) {
salary = s;
}
// Getter
```

Output:

```
50000
```

Example explained

The salary attribute is private , which have restricted access.

The public setSalary() method takes a parameter (s) and assigns it to the salary

attribute (salary = s).

The public getSalary() method returns the value of the private salary attribute.

Inside main() , we create an object of the Employee class. Now we can use the setSalary() method to set the value of the private

attribute to 50000. Then we call the getSalary() method on the object to return the value.

Why Encapsulation?

It is considered good practice to declare your class attributes as private (as often as you can). Encapsulation ensures better control of

your data, because you (or others) can change one part of the code without affecting other parts

« Increased security of data

9.4 Inheritance

In C++, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

e derived class (child) - the class that inherits from another class

« base class (parent) - the class being inherited from

To inherit from a class, use the : symbol.

In the example below, the Car class (child) inherits the attributes and methods from the Vehicle class (parent):

Inheritance

In C++, it is possible to inherit attributes and methods from one class to another. We

group the "inheritance concept" into two categories:

« derived class (child) - the class that inherits from another class

* base class (parent) - the class being inherited from

To inherit from a class, use the : symbol.

In the example below, the Car class (child) inherits the attributes and methods from the Vehicle class (parent):

Example

```
// Base class
class Vehicle {
public:
string brand = "Ford";
void honk() {
cout << "Tuut, tuut! \n" 5
}
}
// Derived class

class Car: public Vehicle {
public:
string model = "Mustang";
}
int main() {
Car myCar;
myCar.honk();
cout << myCar.brand +" " + myCar.model;
return 0;
}
```

Why And When To Use "Inheritance"?

- It is useful for code reusability: reuse attributes and methods of an existing class when you create a new class.

C++ Multilevel Inheritance

Multilevel Inheritance

A class can also be derived from one class, which is already derived from another class.

In the following example, MyGrandChild is derived from class MyChild (which is derived from MyClass).

Example

```
// Base class (parent)
class MyClass {
public:
void myFunction() {
cout << "Some content in parent class." ;
}
}
// Derived class (child)
class MyChild: public MyClass {
}
// Derived class (grandchild)
class MyGrandChild: public MyChild {
```

output:

```
}
```

```
int main() {
MyGrandChild myObj;
myObj .myFunction();
return 0;

}
```

Multiple Inheritance

A class can also be derived from more than one base class, using a comma-separated list:

Example

```
// Base class
class MyClass {
public:
void myFunction() {
cout << "Some content in parent class." ;
}
}
// Another base class
class MyOtherClass {
public:
void myOtherFunction() {
cout << "Some content in another class." ;
}
}
// Derived class
```

```
class MyChildClass: public MyClass, public MyOtherClass {
}
int main() {
MyChildClass myObj;
myObj.myFunction();

myObj .myOtherFunction() ;
return 0;
+
```

Output:

Some content in parent class.

Some content in another class.

Access Specifiers

There are three specifiers available in C++. Until now, we have only used public (members of a class are accessible from outside the class) and private (members can only be accessed within the class). The third specifier, protected, is similar to private, but it can also be accessed in the inherited class:

Example

```
// Base class
class Employee {
protected: // Protected access specifier
int salary;
}
// Derived class
class Programmer: public Employee {
```

```
public:
int bonus;
void setSalary(int s) {
salary = s;
}
int getSalary() {
return salary;
}
}
int main() {
Programmer myObj;
myObj.setSalary (50000) ;
myObj.bonus = 15000;
cout << "Salary: " << myObj.getSalary() << "\n";
cout << "Bonus: " << myObj.bonus << "\n";
return 0;
}
```

9.5 Polymorphism

Polymorphism

Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.

Like we specified in the previous chapter; Inheritance lets us inherit attributes and methods from another class. Polymorphism uses

those methods to perform different tasks. This allows us to perform a single action in different ways.

For example, think of a base class called Animal that has a method called animalSound() . Derived classes of Animals could be Pigs, Cats, Dogs, Birds - And they also have their own implementation of an animal sound (the pig oinks, and the cat meows, etc.):

Example

```
// Base class
class Animal {
public:
void animalSound() {
cout << "The animal makes a sound \n";
}
}
// Derived class
class Pig : public Animal {
public:
void animalSound() {
cout << "The pig says: wee wee \n";
}
}
// Derived class

class Dog : public Animal {
public:
   void animalSound() {
      cout << "The dog says: bow wow \n";
   }
};
```

Now we can create Pig and Dog objects and override the animalsound() method:

Example

```
// Base class
class Animal {
public:
void animalSound() {
cout << "The animal makes a sound \n";
}
}
// Derived class
class Pig : public Animal {
public:
void animalSound() {
cout << "The pig says: wee wee \n";
}
}
// Derived class
class Dog : public Animal {
public:
void animalSound() {
cout << "The dog says: bow wow \n";
}
}
int main() {
Animal myAnimal;
```

```
  Pig myPig;
  Dog myDog;

  myAnimal.animalSound();
  myPig.animalSound();
  myDog.animalSound();
  return 0;
}
```

Output:

The animal makes a sound

The pig says: wee wee

The dog says: bow wow

Why And When To Use "Inheritance" and "Polymorphism"?

- It is useful for code reusability: reuse attributes and methods of an existing class when you create a new class.

Chapter 1 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. Define C++ and explain why it is described as a general-purpose programming language.

_____

_____

2. Name the four object-oriented programming concepts introduced in this chapter.

_____

_____

3. Distinguish between a compiler and an interpreter.

_____

_____

4. What is an IDE? Give two examples mentioned in the chapter.

_____

_____

5. List the main steps required to create and save a first C++ source file in Code::Blocks.

_____

_____

6. Differentiate among syntax errors, run-time errors, and logical errors.

_____

_____

7. Why must source code be translated before a computer can execute it?

_____

_____

8. State one advantage of using a compiler and one advantage of using an interpreter.

_____

_____

Practice and Programming Exercises

1. Write a simple C++ program that prints your name, department, and university on separate lines.

_____

_____

_____

_____

2. Write a C++ program that prints the message “Welcome to C++ Programming”. Identify the source file extension used.

_____

_____

_____

_____

3. Create an example containing a missing semicolon. State the type of error and then correct it.

_____

_____

_____

_____

4. Create an example that attempts division by zero. Explain when the error occurs.

_____

_____

_____

_____

5. Write a program with a logical error in a simple arithmetic expression, then provide the corrected expression.

_____

_____

_____

_____

6. Draw or describe the Problem Analysis–Coding–Execution Cycle in your own words.

_____

_____

_____

_____

7. Prepare a short comparison table containing: compiler, interpreter, source code, and machine code.

_____

_____

_____

_____

8. Install or open a C++ IDE and document the steps used to compile and run a small program.

_____

_____

_____

_____

Chapter 2 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What is a token in C++?

_____

_____

2. List the major categories of C++ tokens presented in this chapter.

_____

_____

3. What rules should be followed when choosing a C++ identifier?

_____

_____

4. Explain the purpose of comments in a program.

_____

_____

5. Differentiate between single-line and multi-line comments.

_____

_____

6. List the basic data types introduced in the chapter and state the kind of value each stores.

_____

_____

7. What is the purpose of the long type modifier?

_____

_____

8. Why are keywords not allowed to be used as identifiers?

_____

_____

Practice and Programming Exercises

1. Classify ten sample items as keywords, identifiers, literals, operators, or punctuation tokens.

_____

_____

_____

_____

2. Write five valid C++ identifiers and five invalid identifiers; explain why each invalid name is not allowed.

_____

_____

_____

_____

3. Write a program containing both single-line and multi-line comments.

_____

_____

_____

_____

4. Declare variables using int, float, double, char, and bool, and assign suitable values.

_____

_____

_____

_____

5. Write a program that displays the size of several basic data types using sizeof.

_____

_____

_____

_____

6. Correct a short code fragment that incorrectly uses a C++ keyword as a variable name.

_____

_____

_____

_____

7. Create declarations for student age, GPA, grade letter, and enrollment status using appropriate data types.

_____

_____

_____

_____

8. Write a short program demonstrating the difference between integer and floating-point values.

_____

_____

_____

_____

Chapter 3 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. Define a variable in C++.

_____

_____

2. What is variable scope?

_____

_____

3. Differentiate between local, formal parameter, and global variables.

_____

_____

4. What is the lifetime of a global variable?

_____

_____

5. Explain the purpose of the const keyword.

_____

_____

6. What is a string in C++?

_____

_____

7. Why is string not considered a fundamental type in the same way as int or char?

_____

_____

8. Explain the purpose of escape sequences such as \n and \t.

_____

_____

Practice and Programming Exercises

1. Declare and initialize variables representing a student ID, age, GPA, and grade.

_____

_____

_____

_____

2. Write a program with a local variable and a global variable having different names, then print both.

_____

_____

_____

_____

3. Write a small function that receives a formal parameter and displays its value.

_____

_____

_____

_____

4. Declare a constant named PI and use it to calculate the area of a circle.

_____

_____

_____

_____

5. Write a program that stores a first name and last name as strings and prints them together.

_____

_____

_____

_____

6. Use \n and \t to format a small student information report.

_____

_____

_____

_____

7. Predict the scope of each variable in a provided nested-block example, then verify by compiling it.

_____

_____

_____

_____

8. Modify a program so that a value that must not change is declared using const.

_____

_____

Chapter 4 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What is an operator in C++?

_____

_____

2. List the operator categories presented in the chapter.

_____

_____

3. Differentiate between arithmetic and relational operators.

_____

_____

4. What values are produced by relational expressions?

_____

_____

5. Explain the logical operators !, &&, and ||.

_____

_____

6. What is the purpose of assignment operators?

_____

_____

7. Describe the role of cin and cout in basic input/output.

_____

_____

8. Why is operator precedence important when evaluating expressions?

_____

_____

Practice and Programming Exercises

1. Write expressions using +, -, *, /, and % and predict their results.

_____

_____

_____

_____

2. Write a program that reads two integers and displays their sum, difference, product, and quotient.

_____

_____

_____

_____

3. Write a program that checks whether one number is greater than another.

_____

_____

_____

_____

4. Create truth tables for && and || using boolean values.

_____

_____

_____

_____

5. Write a program that reads a student mark and evaluates whether it is at least 50.

_____

_____

_____

_____

6. Rewrite a multi-step assignment using a compound assignment operator where appropriate.

_____

_____

_____

_____

7. Write a program that reads length and width and calculates rectangle area and perimeter.

_____

_____

_____

_____

8. Evaluate several mixed arithmetic expressions manually, then confirm the results using C++.

_____

_____

_____

Chapter 5 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What is a control structure?

_____

_____

2. Explain the purpose of selection statements.

_____

_____

3. Differentiate between if, if-else, and multi-branch selection.

_____

_____

4. What is the role of loops in a program?

_____

_____

5. When is a for loop commonly appropriate?

_____

_____

6. When may a while loop be preferable?

_____

_____

7. What is an exception in C++?

_____

_____

8. Explain the roles of try, throw, and catch in exception handling.

_____

_____

Practice and Programming Exercises

1. Write a program that determines whether an integer is positive, negative, or zero.

_____

_____

_____

_____

2. Write a program that prints the numbers 1 through 10 using a for loop.

_____

_____

_____

_____

3. Rewrite the previous exercise using a while loop.

_____

_____

_____

_____

4. Write a program that calculates the sum of integers from 1 to n.

_____

_____

_____

_____

5. Write a program that reads a mark and prints Pass or Fail using if-else.

_____

_____

_____

_____

6. Create a menu-driven program using a suitable selection structure.

_____

_____

_____

_____

7. Write a program that safely handles an attempted division by zero using exception handling.

_____

_____

_____

_____

8. Trace a nested control structure for a given set of input values and record the executed statements.

_____

_____

Chapter 6 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What is a function in C++?

_____

_____

2. Why are functions useful in program design?

_____

_____

3. Differentiate between a function declaration, definition, and call.

_____

_____

4. What is a return value?

_____

_____

5. What is the difference between a parameter and an argument?

_____

_____

6. Explain how functions support code reuse.

_____

_____

7. What does a void function return?

_____

_____

8. Why is decomposition into small functions useful for testing and maintenance?

_____

_____

Practice and Programming Exercises

1. Write a function that returns the square of an integer.

_____

_____

_____

_____

2. Write a function that receives two numbers and returns the larger value.

_____

_____

_____

_____

3. Write a void function that prints a simple course heading.

_____

_____

_____

_____

4. Write a function that calculates the area of a rectangle from two parameters.

_____

_____

_____

_____

5. Create a program that calls the same function several times using different arguments.

_____

_____

_____

_____

6. Write a function that determines whether a number is even.

_____

_____

_____

_____

7. Decompose a small calculator program into separate functions for arithmetic operations.

_____

_____

_____

_____

8. Trace the order of function calls in a program containing main and two user-defined functions.

_____

_____

Chapter 7 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What is an array?

_____

_____

2. Why are arrays useful when several values have the same data type?

_____

_____

3. How are array elements indexed in C++?

_____

_____

4. What is the index of the first element?

_____

_____

5. How can a loop be used to process an array?

_____

_____

6. What problem occurs when an invalid array index is accessed?

_____

_____

7. Differentiate between declaring and initializing an array.

_____

_____

8. Explain how an array can be used to store a collection of student marks.

_____

_____

Practice and Programming Exercises

1. Declare an integer array containing five values and print all elements.

_____

_____

_____

_____

2. Read five student marks into an array and calculate their sum.

_____

_____

_____

_____

3. Extend the previous exercise to calculate the average mark.

_____

_____

_____

_____

4. Find the largest value in an integer array.

_____

_____

_____

_____

5. Find the smallest value in an integer array.

_____

_____

_____

_____

6. Count how many values in an array are even.

_____

_____

_____

_____

7. Reverse the order in which an array is displayed.

_____

_____

_____

_____

8. Write a program that searches an array for a value entered by the user and reports whether it was found.

_____

_____

Chapter 8 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What is file I/O?

_____

_____

2. Why is file storage useful compared with temporary console data?

_____

_____

3. Name the stream classes commonly used for file input and output.

_____

_____

4. What should a program do before reading from or writing to a file?

_____

_____

5. Why should a file be closed after use?

_____

_____

6. What is meant by reading data from a file?

_____

_____

7. What is meant by writing data to a file?

_____

_____

8. Give two examples of information that a student program could store in a file.

_____

_____

Practice and Programming Exercises

1. Write a program that creates a text file and writes a short message to it.

_____

_____

_____

_____

2. Write a program that reads and displays the contents of a text file.

_____

_____

_____

_____

3. Write three student names to a file, one name per line.

_____

_____

_____

_____

4. Read a sequence of integers from a file and calculate their sum.

_____

_____

_____

_____

5. Modify a file-output program to store a student ID and mark.

_____

_____

_____

_____

6. Add a check that reports when an input file cannot be opened.

_____

_____

_____

_____

7. Write a program that copies text from one file to another.

_____

_____

_____

_____

8. Design a small program that stores several course names in a file and then reads them back.

_____

_____

Chapter 9 Summary and Exercises

Chapter Review: Review the key concepts, terminology, examples, and C++ code presented in this chapter before completing the following activities. The exercises are intended for individual practice, laboratory work, and classroom discussion.

Review Questions

1. What does OOP stand for?

_____

_____

2. Differentiate between procedural programming and object-oriented programming.

_____

_____

3. Define a class and an object.

_____

_____

4. What are attributes and methods?

_____

_____

5. What is a constructor?

_____

_____

6. Explain encapsulation and its purpose.

_____

_____

7. Differentiate among public, private, and protected access specifiers.

_____

_____

8. Explain inheritance and give a simple real-world example.

_____

_____

9. What is polymorphism?

_____

_____

10. How can OOP support reuse and maintainability?

_____

_____

Practice and Programming Exercises

1. Create a class Student with attributes for name and ID and a method that displays them.

_____

_____

_____

_____

2. Create two objects from the Student class and assign different values.

_____

_____

_____

_____

3. Add a constructor to initialize the Student attributes.

_____

_____

_____

_____

4. Change one attribute to private and provide public methods to access it appropriately.

_____

_____

_____

_____

5. Create a base class Person and a derived class Student.

_____

_____

_____

_____

6. Create a base class Shape and derived classes that demonstrate inherited behavior.

_____

_____

_____

_____

7. Write a small example that demonstrates protected access in an inherited class.

_____

_____

_____

_____

8. Create two related classes that demonstrate polymorphic behavior.

_____

_____

_____

_____

9. Compare a procedural solution and an object-oriented solution for representing student records.

_____

_____

_____

_____

10. Design a class for a real-world object of your choice, listing its attributes, methods, constructor, and access specifiers.

_____

_____

_____

_____

References

https://www.amazon.com/Beginning-Game-Programming-scratch-building/dp/1835081746/ref=zg_bs_g_9045760011_d_sccl_6/143-7686813-7413349?psc=1

https://www.w3schools.com/cpp/default.asp

https://www.amazon.com/C-Crash-Course-Josh-Lospinoso/dp/1593278888/ref=zg_bs_g_9045760011_d_sccl_4/143-7686813-7413349?psc=1

Tselikis, G. S. (2022). *Introduction to C++*. CRC Press.

Malik, D. S. (2023). *C++ programming*. Cengage Learning, EMEA.